Solution: Factorial

Problem Statement

Calculate the Factorial of a Positive Number Using Recursion.

The factorial of a non-negative integer N, denoted as N!, is the product of all positive integers less than or equal to N. The factorial of 0 is defined as 1.

Here is what the example input/output looks like:

Input	Expected Output	Explanation
Number = 5	Factorial = 120	The factorial of 5 is calculated as 5 _ 4 _ 3 _ 2 _ 1 = 120.
Number = 7	Factorial = 5040	The factorial of 7 is calculated as 7 _ 6 _ 5 _ 4 _ 3 _ 2 _ 1 = 5040.
Number = 1	Factorial = 1	The factorial of 1 is 1 itself.

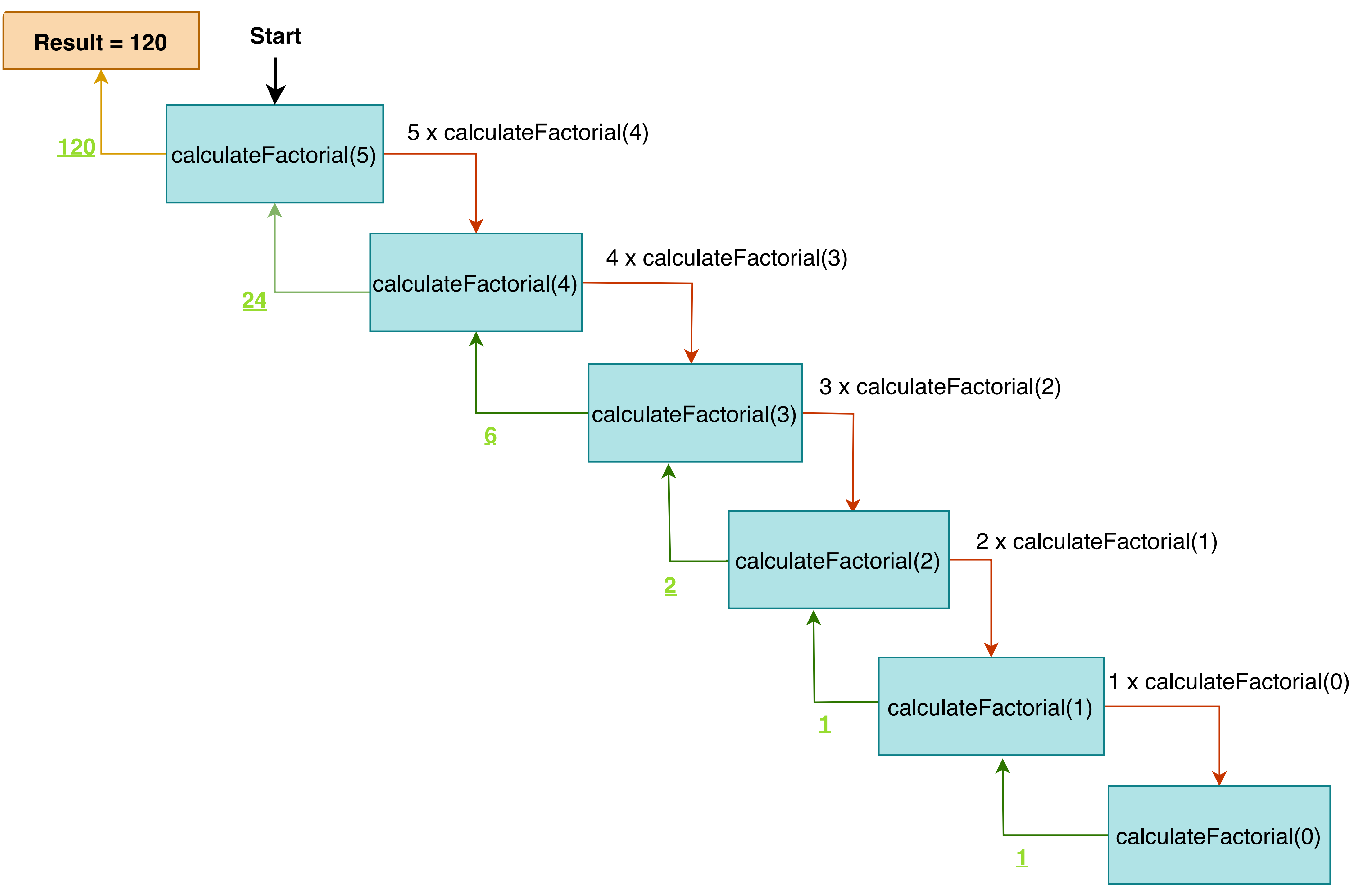
Solution

The algorithm to calculate the factorial of a positive number using recursion can be defined as follows:

1. If the given number is 0 or 1, return 1 (base case).
2. Otherwise, return the product of the number and the recursive call to calculate the factorial of the number minus 1.

The recursive approach works by breaking down the problem into smaller subproblems. We calculate the factorial of a number by multiplying it with the factorial of the number minus 1. This process continues until the base case is reached, where the number is 0 or 1. The base case stops the recursion and returns 1.

By multiplying the numbers in decreasing order, we ensure that each number is multiplied with the product of the previous numbers, eventually giving us the factorial of the given number.



Example dry run to calculate Factorial of 5

Code

Here is the code for this algorithm:

Java

```java
public class Solution {
    public static int calculateFactorial(int number) {
        if (number == 0 || number == 1) {
            return 1; // Base case
        }
        return number * calculateFactorial(number - 1); // Recursive call
    }

    public static void main(String[] args) {
        // Example inputs
        int[] examples = {5, 7, 1};

        for (int number : examples) {
            int factorial = calculateFactorial(number);
            System.out.println("Factorial of " + number + ": " + factorial);
        }
    }
}

```

Run Code Result Stdout

You must run your code first

Execute

Time and Space Complexity

The time complexity of the algorithm is $O(N)$ since we make N recursive calls, where N is the given input.

The space complexity of the algorithm is also $O(N)$ because the maximum depth of the recursion is N. Each recursive call adds a new stack frame, consuming additional space on the call stack.

← Previous
2. Factorial

Next →
3. Greatest Common Divisor (GCD)

☐ Mark as Completed